

Лабораторная работа №1. Классификация изображений с применением
Transfer Learning.

Выполнил:

Доценников Никита

Группа К3321

Проверил:

Дмитрий Владимирович Царёв

Содержание

Введение	3
Репозиторий с исходным кодом	3
Основная часть	4
Предварительная обработка данных	4
Кодирование меток	7
Логистический классификатор	8
Наивный байесовский классификатор	10
Матрица неточностей	13
Машины опорных векторов	15
Регрессор на основе одной переменной	17
Многомерный регрессор	19
Оценка стоимости жилья с помощью регрессора на опорных векторах	21
Заключение	23

Введение

Целью данной работы является получение практических навыков по обработке данных, кластеризации и классификации с помощью алгоритмов обучения с учителем, реализованных в библиотеке Sklearn.

В рамках работы решаются следующие задачи. Выполняется предварительная обработка числовых и текстовых данных, включая бинаризацию, исключение среднего, масштабирование и нормализацию. Реализуется кодирование текстовых меток в числовую форму. Строится логистический классификатор и исследуется влияние параметра регуляризации на его работу. Реализуется наивный байесовский классификатор с оценкой качества по метрикам точности, полноты и меры F1. Строится матрица неточностей и формируется отчёт о классификации. Создаётся классификатор на основе машины опорных векторов для прогнозирования уровня дохода человека. Реализуются одномерная, многомерная и полиномиальная регрессии. Строится регрессор на основе машины опорных векторов для оценки стоимости жилья.

Используемые инструменты, среда Python версии 3.14, библиотека scikit learn, библиотека matplotlib для визуализации результатов.

Репозиторий с исходным кодом

Исходный код всех программ, рассматриваемых в пунктах 1–8 данной работы, размещён в репозитории.

[Ссылка на репозиторий с исходным кодом](#)

Основная часть

Предварительная обработка данных

Для демонстрации методов предварительной обработки использовалась небольшая выборка числовых данных, состоящая из четырёх строк по три признака. К данным последовательно были применены четыре метода, а именно бинаризация по пороговому значению 2.1, исключение среднего, масштабирование в диапазон от 0 до 1 и нормализация по нормам L1 и L2.

```
input_data = np.array(  
    [[5.1, -2.9, 3.3],  
     [-1.2, 7.8, -6.1],  
     [3.9, 0.4, 2.1],  
     [7.3, -9.9, -4.5]]  
)  
  
data_binarized = pre.Binarizer(threshold=2.1).transform(input_data)  
data_scaled = pre.scale(input_data)  
data_scaler_minmax = pre.MinMaxScaler(feature_range=(0, 1))  
data_scaled_minmax = data_scaler_minmax.fit_transform(input_data)  
data_normalized_l1 = pre.normalize(input_data, norm="l1")  
data_normalized_l2 = pre.normalize(input_data, norm="l2")
```

```

Binarized data:
[[1. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [1. 0. 0.]]

BEFORE:
Mean= [ 3.775 -1.15 -1.3 ]
Std deviation = [3.12039661 6.36651396 4.0620192 ]

AFTER:
Mean = [1.11022302e-16 0.00000000e+00 2.77555756e-17]
Std deviation = [1. 1. 1.]

Min max scaled data:
[[0.74117647 0.39548023 1.
 0.
 1.
 0.]
 [0.6
 0.5819209
 0.87234043]
 [1.
 0.
 0.17021277]]

L1 normalized data:
[[ 0.45132743 -0.25663717 0.2920354 ]
 [-0.0794702 0.51655629 -0.40397351]
 [ 0.609375 0.0625 0.328125 ]
 [ 0.33640553 -0.4562212 -0.20737327]]

L2 normalized data:
[[ 0.75765788 -0.43082507 0.49024922]
 [-0.12030718 0.78199664 -0.61156148]
 [ 0.87690281 0.08993875 0.47217844]

```

Рис. 1. Терминальный вывод предварительной обработки данных

После бинаризации все значения, превышающие пороговое значение 2.1, заменяются единицей, а остальные значения заменяются нулём. После стандартизации среднее значение каждого признака становится близким к нулю, а стандартное отклонение равным единице, что подтверждается выводом программы. Масштабирование MinMax приводит значения каждого столбца к диапазону от 0 до 1, при этом минимальное значение признака отображается в 0, а максимальное в 1. Нормализация L1 обеспечивает равенство единице суммы модулей значений в каждой строке, а нормализация L2

обеспечивает равенство единице суммы квадратов значений. Нормализация L1 считается более устойчивой к выбросам, поскольку крупные отклонения влияют на сумму модулей слабее, чем на сумму квадратов при использовании нормы L2.

Кодирование меток

Для демонстрации кодирования меток использовался набор из семи текстовых меток цветов. Кодировщик `LabelEncoder` был обучен на этом наборе, после чего было выполнено прямое преобразование меток в числа и обратное преобразование чисел в исходные текстовые метки.

```
input_labels = ['red', 'black', 'red', 'green', 'black', 'yellow',
               'white']

encoder = pre.LabelEncoder()
encoder.fit(input_labels)

test_labels = ['green', 'red', 'black']
encoded_values = encoder.transform(test_labels)

encoded_values = [3, 0, 4, 1]
decoded_list = encoder.inverse_transform(encoded_values)
```

```
Label mapping:
black --> 0
green --> 1
red --> 2
white --> 3
yellow --> 4

Labels = ['green', 'red', 'black']
Encoded values = [np.int64(1), np.int64(2), np.int64(0)]

Encoded values = [3, 0, 4, 1]
Decoded labels = [np.str_('white'), np.str_('black'), np.str_('yellow'), np.str_('green')]
```

Рис. 2. Терминальный вывод кодирования и декодирования меток

Кодировщик присвоил меткам числа в алфавитном порядке, а именно black соответствует 0, green соответствует 1, red соответствует 2, white соответствует 3, yellow соответствует 4. Метки green, red, black были корректно преобразованы в числа 1, 2, 0. Обратное преобразование чисел 3, 0, 4, 1 дало исходные метки white, black, yellow, green, что подтверждает корректность работы кодировщика в обе стороны.

Логистический классификатор

Логистический классификатор был обучен на выборке из двенадцати двумерных точек, разделённых на четыре класса. Классификатор строился с параметром регуляризации C , значение которого определяет штраф за ошибки классификации на обучающей выборке. Работа классификатора была протестирована при двух значениях данного параметра, C равном 1 и C равном 100.

```
classifier = linear_model.LogisticRegression(solver='lbfgs', C=1)
classifier.fit(X, y)
visualize_classifier(classifier, X, y)
```

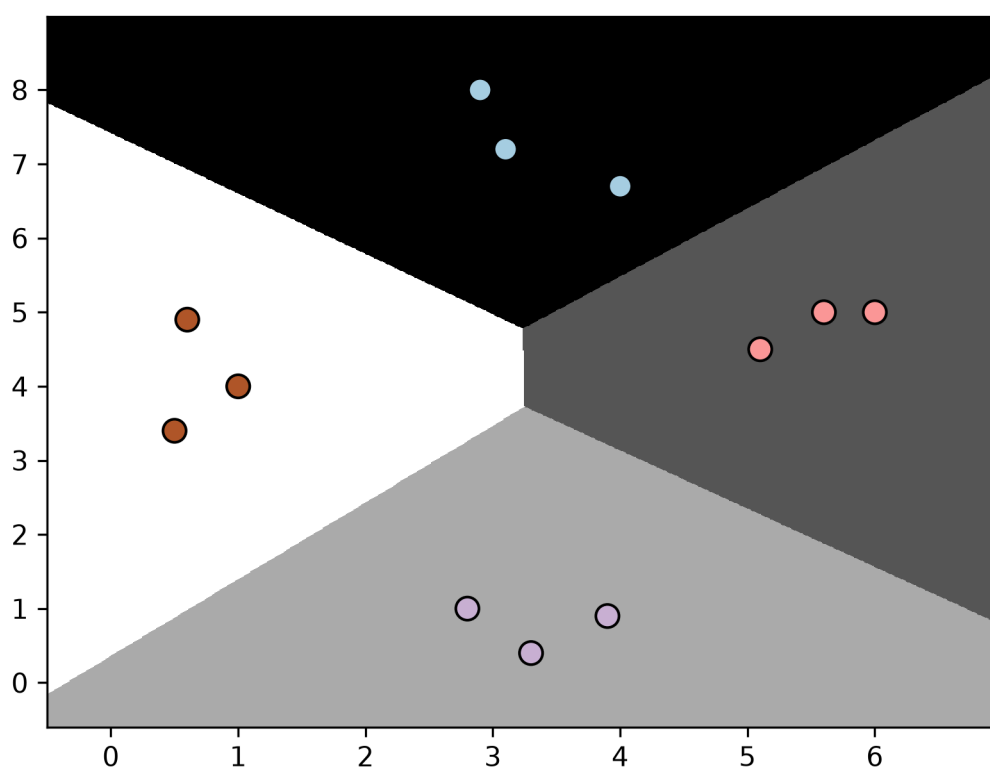


Рис. 3. Границы классов логистического классификатора при C равном 1

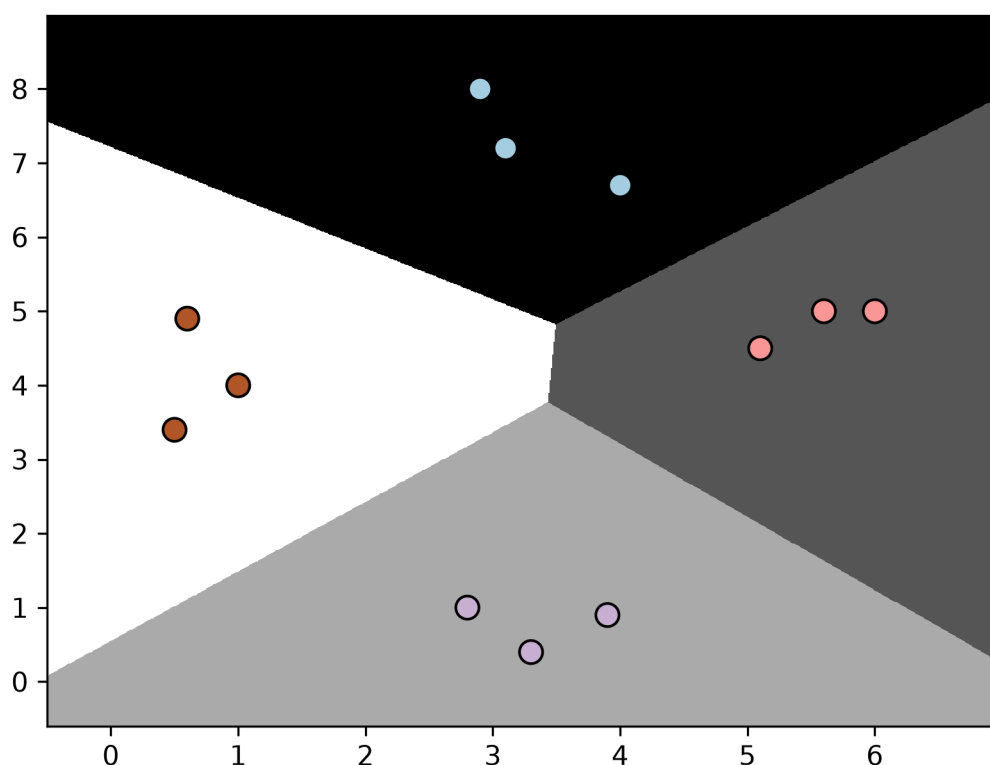


Рис. 4. Границы классов логистического классификатора при C равном 100

При малом значении параметра C регуляризация сильнее ограничивает модель, поэтому положение разделяющих прямых определяется в большей степени общим расположением точек, а не отдельными выбросами. При увеличении C до 100 штраф за ошибки классификации возрастает, и модель сильнее подстраивает положение и наклон разделяющих прямых под конкретное расположение обучающих точек, поскольку регуляризация слабее сдерживает величину коэффициентов. Так как логистическая регрессия строит только линейные границы, визуальная разница между двумя значениями C может быть небольшой для хорошо разделимой выборки, однако при менее чистых данных увеличение C обычно приводит к более резкой подстройке модели под обучающую выборку и повышает риск переобучения.

Наивный байесовский классификатор

Наивный байесовский классификатор `GaussianNB` был обучен на данных из файла `data_multivar_nb.txt`. Сначала модель была обучена и протестирована на всей выборке целиком, затем данные были разбиты на обучающую и тестовую части в пропорции 80 к 20 для получения более объективной оценки качества.

```
classifier = GaussianNB()
classifier.fit(X, y)
y_pred = classifier.predict(X)
accuracy = 100.0 * (y == y_pred).sum() / X.shape[0]
```

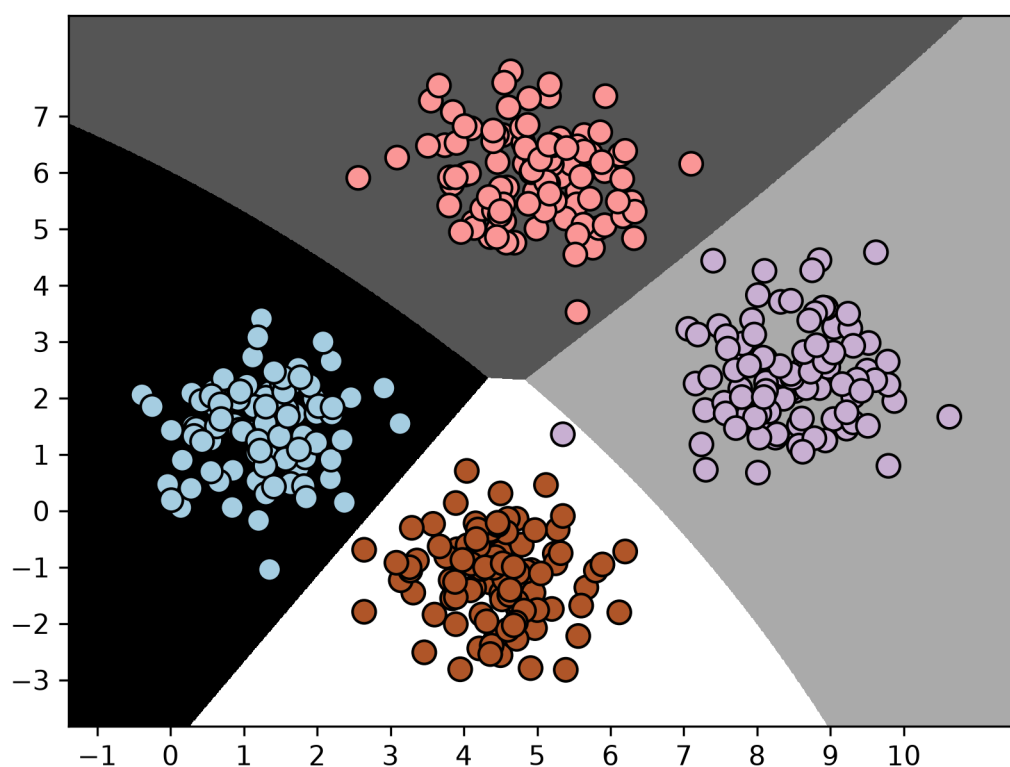


Рис. 5. Границы классов наивного байесовского классификатора на полной выборке

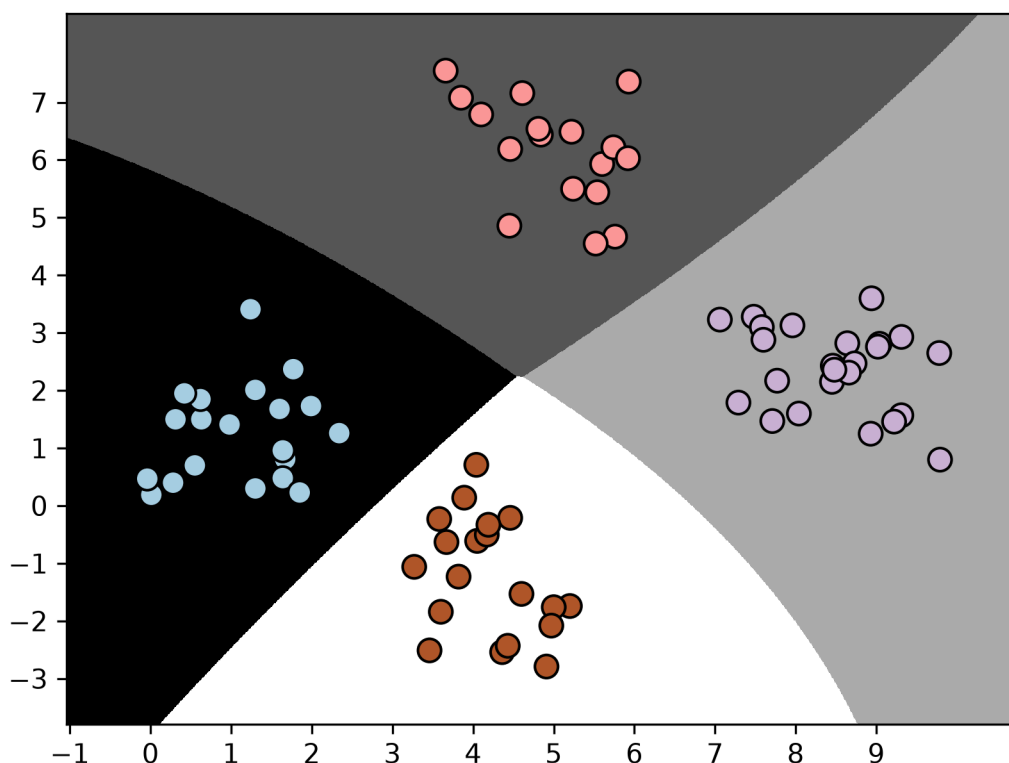


Рис. 6. Границы классов наивного байесовского классификатора на тестовой выборке

```
Accuracy of Naive Bayes classifier = 99.75 %
Accuracy of the new classifier = 100.0 %
Accuracy: 99.75%
Precision: 99.76%
Recall: 99.75%
F1: 99.75%
```

Рис. 7. Терминальный вывод метрик наивного байесовского классификатора

При обучении и тестировании на одной и той же полной выборке точность классификатора составила 99.75 процента. При разбиении данных на обучающую и тестовую части точность на новой, независимой тестовой выборке составила 100 процентов. Такой результат объясняется тем, что классы в используемом наборе данных хорошо разделимы и имеют распределения, близкие к нормальному, что соответствует базовому предположению гауссовского наивного байесовского классификатора. Трёхкратная перекрёстная проверка дала более устойчивую оценку качества, значение точности (ассигасу) составило 99.75 процента, точность (precision) 99.76 процента, полнота (recall) 99.75 процента, мера F1 99.75 процента. Близость всех четырёх метрик друг к другу говорит о том, что классификатор одина-

ково хорошо справляется с обнаружением объектов всех классов, не отдавая предпочтения ни одному из них.

Матрица неточностей

Для построения матрицы неточностей использовались заранее заданные списки истинных и предсказанных меток из пяти классов. На основе этих списков была построена матрица неточностей и сформирован отчёт о классификации с метриками точности, полноты и меры F1 по каждому классу.

```
true_labels = [2, 0, 0, 2, 4, 4, 1, 0, 3, 3, 3]
pred_labels = [2, 1, 0, 2, 4, 3, 1, 0, 1, 3, 3]

confusion_mat = confusion_matrix(true_labels, pred_labels)
targets = ['Class-0', 'Class-1', 'Class-2', 'Class-3', 'Class-4']
print(classification_report(true_labels, pred_labels,
target_names=targets))
```

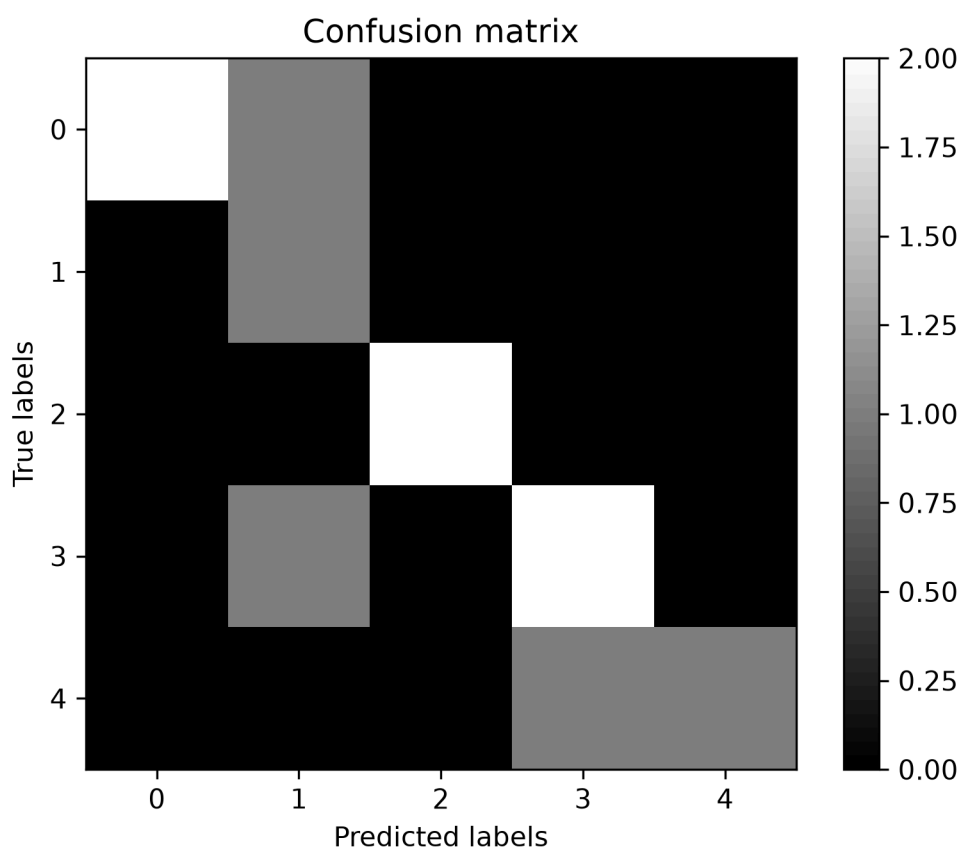


Рис. 8. Визуализация матрицы неточностей

	precision	recall	f1-score	support
Class-0	1.00	0.67	0.80	3
Class-1	0.33	1.00	0.50	
Class-2	1.00	1.00	1.00	
Class-3	0.67	0.67	0.67	
Class-4	1.00	0.50	0.67	
accuracy			0.73	
macro avg	0.80	0.77	0.73	
weighted avg	0.85	0.73	0.75	11

Рис. 9. Терминальный вывод отчёта о классификации

По результатам отчёта о классификации общая точность составила 0.73. Наилучшие показатели наблюдаются для класса Class-2, для которого точность, полнота и мера F1 равны единице, поскольку оба объекта этого класса были классифицированы верно. Наихудшие показатели наблюдаются для класса Class-1, точность которого составила лишь 0.33 при полноте, равной единице. Это означает, что классификатор верно находит все объекты, относящиеся к данному классу, но при этом ошибочно относит к этому же классу значительное количество объектов, принадлежащих другим классам, что видно из матрицы неточностей по единичным значениям вне диагонали в столбце, соответствующем Class-1.

Машины опорных векторов

Для решения задачи бинарной классификации по прогнозированию уровня дохода человека использовался классификатор на основе линейной машины опорных векторов, обёрнутый в схему один против одного. Каждый признак данных, являющийся текстовым, был предварительно закодирован собственным экземпляром LabelEncoder, а числовые признаки использовались без изменений.

```
classifier = OneVsOneClassifier(LinearSVC(random_state=0))
classifier.fit(X_train, y_train)

f1 = model_selection.cross_val_score(classifier, X, y,
scoring='f1_weighted', cv=3)
print("F1 score: " + str(round(100 * f1.mean(), 2)) + "%")
```



F1 score: 76.09%

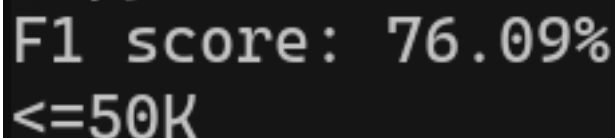
Рис. 10. Терминальный вывод меры F1 для классификатора на опорных векторах

Ниже приведены три различные тестовые точки данных, для которых требуется получить прогноз классификатора.

```
input_data_1 = ['37', 'Private', '215646', 'HS-grad', '9', 'Never-
married',
                'Handlers-cleaners', 'Not-in-family', 'White', 'Male',
                '0', '0', '40', 'United-States']

input_data_2 = ['42', 'Private', '198183', 'Bachelors', '13',
'Married-civ-spouse',
                'Exec-managerial', 'Husband', 'White', 'Female',
                '5178', '0', '50', 'United-States']

input_data_3 = ['23', 'Private', '122272', '11th', '7', 'Never-
married',
                'Sales', 'Own-child', 'Black', 'Male',
                '0', '0', '25', 'United-States']
```



F1 score: 76.09%
<=50K

Рис. 11. Прогноз уровня дохода для тестовой точки 1

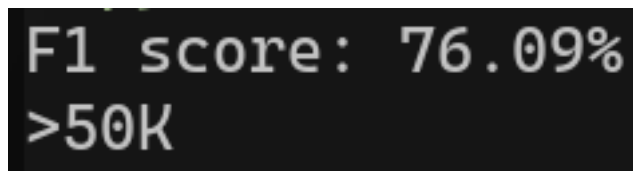
A black rectangular box with white text. The text is arranged in two lines: the first line reads "F1 score: 76.09%" and the second line reads ">50K".

Рис. 12. Прогноз уровня дохода для тестовой точки 2

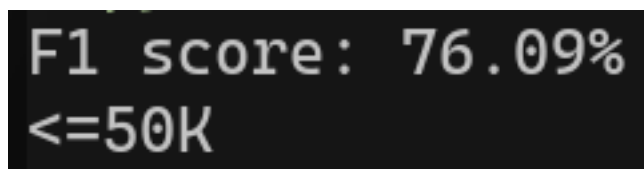
A black rectangular box with white text. The text is arranged in two lines: the first line reads "F1 score: 76.09%" and the second line reads "<=50K".

Рис. 13. Прогноз уровня дохода для тестовой точки 3

Полученная мера F1 при трёхкратной перекрёстной проверке составила 76.09 процента, что заметно ниже показателей наивного байесовского классификатора из предыдущего раздела. Такой результат объясняется тем, что реальный набор данных о доходах содержит гораздо более сложные и частично пересекающиеся зависимости между признаками по сравнению с искусственно сгенерированными выборками, а линейная граница, которую способен построить LinearSVC, не всегда способна точно отразить эти зависимости. Для первой тестовой точки, соответствующей человеку с частной занятостью, средним образованием и стандартной рабочей неделей, классификатор предсказал класс дохода не более 50 тысяч долларов в год, что согласуется с характеристиками точки. Для второй тестовой точки, отличающейся более высоким уровнем образования (степень бакалавра), руководящей должностью, официальным браком и большим числом отработанных часов в неделю, классификатор предсказал класс дохода свыше 50 тысяч долларов в год, что также хорошо согласуется со здравым смыслом, поскольку перечисленные признаки традиционно связаны с более высоким доходом. Для третьей точки, соответствующей молодому человеку без семьи, с неполным средним образованием и минимальным трудовым стажем, классификатор вновь предсказал класс дохода не более 50 тысяч долларов в год. Таким образом, классификатор оказался чувствителен к ключевым признакам, определяющим уровень дохода, а именно к уровню образования, семейному положению и количеству отработанных часов, что подтверждает содержательность построенной модели, несмотря на умеренное значение меры F1.

Регрессор на основе одной переменной

Для построения одномерного линейного регрессора использовались данные из файла `data_singlevar_regr.txt`. Выборка была разбита на обучающую и тестовую части в пропорции 80 к 20, после чего была обучена модель линейной регрессии, а результаты предсказания на тестовой части были визуализированы и сохранены на диск с помощью модуля `pickle`.

```
regressor = linear_model.LinearRegression()  
regressor.fit(X_train, y_train)  
y_test_pred = regressor.predict(X_test)
```

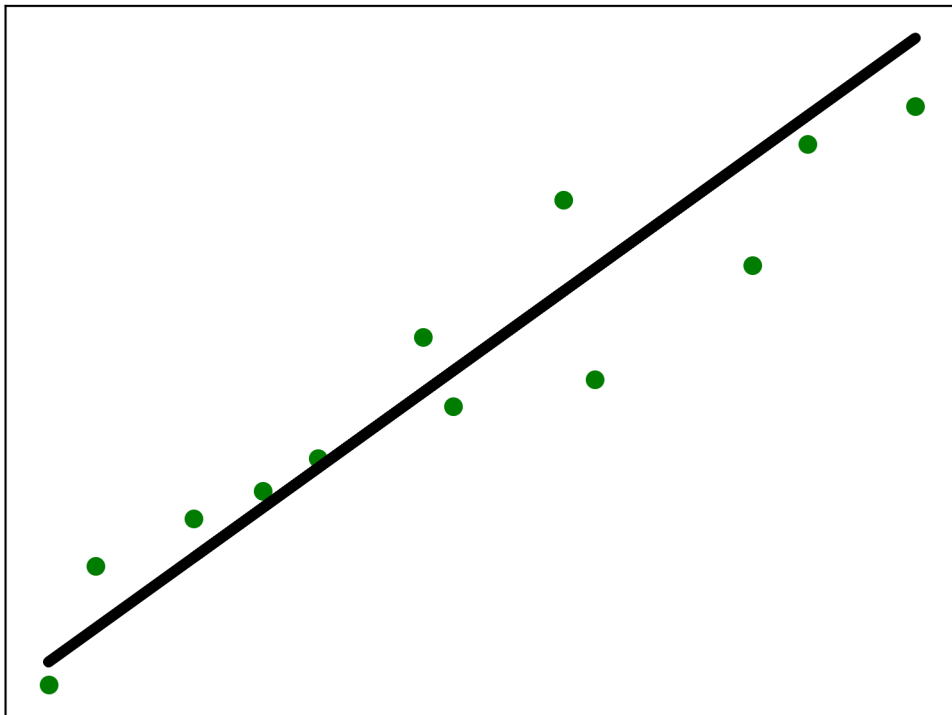


Рис. 14. График прогноза одномерного линейного регрессора

```
Linear regressor performance:  
Mean absolute error = 0.59  
Mean squared error = 0.49  
Median absolute error = 0.51  
Explain variance score = 0.86  
R2 score = 0.86  
  
New mean absolute error = 0.59
```

Рис. 15. Терминальный вывод метрик одномерного регрессора

По результатам работы регрессора средняя абсолютная ошибка составила 0.59, среднеквадратичная ошибка 0.49, медианная абсолютная ошибка 0.51, объяснённая дисперсия 0.86, коэффициент детерминации R2 также составил 0.86. Значение R2, равное 0.86, говорит о том, что построенная линейная модель объясняет около 86 процентов дисперсии целевой переменной, что можно считать хорошим результатом для одномерной регрессии. После сохранения модели в файл и её повторной загрузки средняя абсолютная ошибка совпала с исходным значением 0.59, что подтверждает корректность сериализации модели с помощью модуля pickle.

Многомерный регрессор

Для построения многомерного регрессора использовались данные из файла `data_multivar_regr.txt`, содержащие несколько признаков. Были построены и сравнены две модели, линейная регрессия и полиномиальная регрессия десятой степени.

```
linear_regressor = linear_model.LinearRegression()
linear_regressor.fit(X_train, y_train)

polynomial = PolynomialFeatures(degree=10)
X_train_transformed = polynomial.fit_transform(X_train)
poly_linear_model = linear_model.LinearRegression()
poly_linear_model.fit(X_train_transformed, y_train)
```

```
Linear Regressor performance:
Mean absolute error = 3.58
Mean squared error = 20.31
Median absolute error = 2.99
Explained variance score = 0.86
R2 score = 0.86
```

```
Linear regression:
[36.05286276]
```

```
Polynomial regression:
[31.63944054]
```

Рис. 16. Терминальный вывод метрик и прогнозов многомерного регрессора

Линейная многомерная регрессия показала среднюю абсолютную ошибку 3.58, среднеквадратичную ошибку 20.31, медианную абсолютную ошибку 2.99, объяснённую дисперсию 0.86 и коэффициент детерминации 0.86. Для тестовой точки данных, близкой по значениям к одной из строк обучающего набора, линейная регрессия предсказала значение около 36.05, тогда как полиномиальная регрессия десятой степени предсказала значение около 31.64. Ожидаемое значение, указанное в задании, составляет около 41.35, поэтому обе модели дали заметное отклонение от эталонного значе-

ния, при этом полиномиальная модель отклонилась сильнее линейной. Такой результат объясняется тем, что полином десятой степени, обученный на малом количестве обучающих точек, легко переобучается и начинает плохо экстраполировать значения даже для точек, находящихся вблизи обучающей выборки, тогда как линейная модель за счёт своей простоты оказывается более устойчивой на этих данных.

Оценка стоимости жилья с помощью регрессора на опорных векторах

Для оценки стоимости жилья использовался набор данных California housing, доступный непосредственно в составе библиотеки scikit learn. Строился регрессор SVR с линейным ядром, параметр C которого задаёт штраф за ошибки на обучающей выборке, а параметр epsilon задаёт ширину полосы, внутри которой ошибка прогноза не штрафруется.

```
sv_regressor = SVR(kernel='linear', C=1.0, epsilon=0.1)
sv_regressor.fit(X_train, y_train)

y_test_pred = sv_regressor.predict(X_test)
mse = mean_squared_error(y_test, y_test_pred)
evs = explained_variance_score(y_test, y_test_pred)
```

Ниже приведены три комбинации параметров и тестовых точек данных.

```
# 1
sv_regressor = SVR(kernel='linear', C=1.0, epsilon=0.1)
test_data_1 = [8.3, 41, 6.98, 1.02, 322, 2.55, 37.88, -122.23]

# 2
sv_regressor = SVR(kernel='linear', C=10.0, epsilon=0.1)
test_data_2 = [3.5, 25, 5.2, 1.1, 1500, 3.0, 34.05, -118.25]

# 3
sv_regressor = SVR(kernel='linear', C=1.0, epsilon=0.5)
test_data_3 = [5.1, 15, 6.0, 1.0, 800, 2.8, 36.6, -121.9]
```

```
Mean squared error = 0.42032031367021444
Explained variance score = 0.6689384630173738

Predicted price: 4.151530060419231
```

Рис. 17. Метрики и прогноз стоимости жилья, комбинация параметров 1

```
Mean squared error = 0.420379711596946
Explained variance score = 0.6689098014574179

Predicted price: 1.8723329210561863
```

Рис. 18. Метрики и прогноз стоимости жилья, комбинация параметров 2

```
Mean squared error = 0.41883860750230073
Explained variance score = 0.6695237357293788

Predicted price: 2.794875207927933
```

Рис. 19. Метрики и прогноз стоимости жилья, комбинация параметров 3

Во всех трёх запусках метрики оказались близки друг к другу, что говорит о том, что модель находится вблизи режима насыщения по параметрам C и ϵ в выбранном диапазоне значений. При увеличении C с 1.0 до 10.0 среднеквадратичная ошибка практически не изменилась, что означает, что усиление штрафа за ошибки на обучающей выборке уже не даёт модели заметного выигрыша, поскольку основным ограничением качества выступает не слабость регуляризации, а сама линейная форма ядра, недостаточно гибкая для описания зависимости цены жилья от восьми исходных признаков. Более заметный, хотя всё ещё небольшой эффект дало увеличение ϵ с 0.1 до 0.5, при котором среднеквадратичная ошибка снизилась с 0.42032 до 0.41884, а объяснённая дисперсия выросла с 0.66894 до 0.66952. Такое направление изменения объясняется тем, что более широкая полоса нечувствительности позволяет модели игнорировать часть небольших, вероятно шумовых отклонений при обучении, за счёт чего итоговое решение получается немного более устойчивым на тестовой выборке. Тем не менее масштаб всех наблюдаемых изменений в пределах тысячных долей показывает, что для существенного улучшения качества модели в данной задаче требуется не столько подбор C и ϵ , сколько переход к нелинейному ядру, способному учитывать более сложные зависимости между признаками.

Заключение

В ходе работы были реализованы основные этапы обучения с учителем в `scikit-learn`, такие как предварительная обработка данных, кодирование меток, построение классификаторов и регрессионных моделей.

Наивный байесовский классификатор показал наилучший результат на синтетических данных с хорошо разделимыми классами. Для логистического классификатора изменение параметра C со 1 до 100 практически не повлияло на границы классов, поскольку данные хорошо разделимы. Классификатор на опорных векторах на реальных данных о доходах показал умеренное качество, что связано со сложностью данных и линейностью границы. Матрица неточностей показала, что общая точность скрывает различия между классами, например, для Class-1 полнота равна 1.00, а точность только 0.33.

Одномерная и многомерная линейные регрессии дали $R^2 = 0.86$. Полиномиальная регрессия десятой степени на контрольной точке оказалась менее точной, чем линейная, что согласуется с риском переобучения. Регрессор на опорных векторах для оценки стоимости жилья показал умеренное качество и практически не зависел от параметров C и `epsilon`.

Таким образом, качество моделей определяется прежде всего свойствами данных и соответствием сложности модели задаче, а подбор отдельных параметров играет меньшую роль.